UNIP — Universidade Paulista

Exercícios — Enunciados originais e Respostas

Nome: Igor [sobrenome conforme preenchido à mão] dos Santos
Prof.: MSc. Mateus de Paula
Data: 03/09/2026
Matrícula: F368009

Observação: o nome do(a) aluno(a) estava manuscrito e de difícil leitura na foto enviada — confira a grafia exata antes de entregar. Os enunciados abaixo foram transcritos literalmente das fotos enviadas (inclusive eventuais erros de digitação do documento original); as respostas estão destacadas em cada questão.

1.

> Uma loja de roupas deseja armazenar dados sobre suas vendas e seus produtos. Ela deve criar um banco de dados ou um SGBD?

Resposta:

A loja deve criar um banco de dados, utilizando um SGBD (Sistema Gerenciador de Banco de Dados) já existente no mercado — como PostgreSQL, MySQL ou SQL Server — para criá-lo e administrá-lo.

O SGBD é o software responsável por criar, armazenar, gerenciar e manipular bancos de dados. O banco de dados é o conjunto estruturado de dados (vendas, produtos, clientes etc.) armazenado dentro do SGBD. Ou seja, a loja não precisa desenvolver um SGBD do zero — ela usa um SGBD pronto para criar o seu banco de dados de vendas e produtos.

2.

> Quais são as etapas do Projeto de banco de dados?

Resposta:

1) Levantamento e análise de requisitos — entender as necessidades dos usuários e do negócio.

2) Projeto conceitual — criação do modelo conceitual (ex.: Diagrama Entidade-Relacionamento), independente de tecnologia.

3) Projeto lógico — tradução do modelo conceitual para o modelo lógico (ex.: modelo relacional, com tabelas, chaves e normalização).

4) Projeto físico — implementação do modelo lógico em um SGBD específico (tipos de dados, índices, armazenamento).

5) Implementação, testes, carga de dados e manutenção contínua do banco.

3.

> No modelo relacional de dados, uma chave primária deve possuir quais propriedades?

Resposta:

• Unicidade: o valor deve ser único para cada tupla (linha) da tabela.

• Não nulidade: nunca pode conter valor nulo (NOT NULL).

• Estabilidade/imutabilidade: o valor não deve ser alterado ao longo do tempo.

• Minimalidade: deve usar o menor número possível de atributos necessários para garantir a unicidade.

4.

> Qual objetivo da criação de chaves estrangeiras no modelo relacional?

Resposta:

Garantir a integridade referencial entre tabelas: uma chave estrangeira faz com que o valor armazenado em uma tabela "filha" corresponda obrigatoriamente a um valor existente na chave primária da tabela "pai" (ou seja nulo, se permitido), evitando referências a registros inexistentes e mantendo a consistência dos relacionamentos do banco.

5.

> Considere o modelo conceitual abaixo:

```
[ Aluno ]  —(0,n)—  ◇ Relacao1 ◇  —(0,n)—  [ Curso ]
```

> Converta este modelo para esquema lógico relacional de banco de dados (obs.: utilize o PostgreSQL).

Resposta:

O relacionamento Aluno (0,n) — Relacao1 — (0,n) Curso é N:N (muitos-para-muitos). Em um modelo relacional, todo relacionamento N:N vira uma tabela associativa própria, cuja chave primária é a combinação das chaves estrangeiras das duas tabelas:

```
CREATE TABLE aluno (
    id_aluno SERIAL PRIMARY KEY,
    nome     VARCHAR(100) NOT NULL
);

CREATE TABLE curso (
    id_curso SERIAL PRIMARY KEY,
    nome     VARCHAR(100) NOT NULL
);

CREATE TABLE relacao1 (
    id_aluno INT REFERENCES aluno(id_aluno),
    id_curso INT REFERENCES curso(id_curso),
    PRIMARY KEY (id_aluno, id_curso)
);
```

6.

> O que são e para que servem os schemas no Postgresql?

Resposta:

Um schema é um espaço de nomes (namespace) dentro de um banco de dados que agrupa e organiza objetos (tabelas, views, funções, sequences etc.). Eles servem para: organizar objetos logicamente (ex.: separar dados de "vendas" e "rh"); permitir que vários usuários/aplicações usem o mesmo banco sem conflito de nomes de objetos; e controlar permissões de acesso de forma granular por schema.

```
CREATE SCHEMA vendas;
CREATE TABLE vendas.pedido (
    id SERIAL PRIMARY KEY,
    valor NUMERIC(10,2)
);
```

7.

> É possível distribuir, modificar e utilizar o postgresql sem custo e sob qual licença?

Resposta:

Sim. O PostgreSQL pode ser distribuído, modificado e utilizado livremente, inclusive para fins comerciais, sem pagamento de royalties. Ele é licenciado sob a PostgreSQL License, uma licença de código aberto permissiva, semelhante às licenças MIT e BSD.

9.

> Faça uma lista de empresas que utilizam o PostgreSQL.

Resposta:

• Apple

• Instagram

• Spotify

• Reddit

• Skype/Microsoft

• Twitch

• IMDb

• Netflix (em parte de sua infraestrutura)

• NASA

• Trivago

10.

> Utilizando o PostgreSQL, crie um banco de dados para resolver as consultas abaixo:
>
> a) Apresente todos os dados dos vendedores
>
> b) Apresente todos os dados dos vendedores organizados pelo nome do vendedor
>
> c) Apresente o nome dos vendedores que possuem a substring 'na' na composição do nome
>
> d) Calcule e mostre a soma das vendas realizadas
>
> e) Calcule e mostre a soma das vendas realizadas agrupadas por vendedor
>
> f) Mostre o nome e a data de nascimento dos vendedores que possuem alguma venda realizada
>
> g) Apresente o nome do vendedor que vendeu o produto 'martelo'.
>
> h) Mostre o total de vendedores cadastrados
>
> i) Atualize a tabela de vendedor, aumentando o salário em 100 reais para aqueles vendedores cujo total das vendas daquele vendedor foi maior que 1000 reais.

Resposta:

Modelagem utilizada:

```
CREATE TABLE vendedor (
    id              SERIAL PRIMARY KEY,
    nome            VARCHAR(100) NOT NULL,
    data_nascimento DATE,
    salario         NUMERIC(10,2)
);

CREATE TABLE venda (
    id           SERIAL PRIMARY KEY,
    id_vendedor  INT REFERENCES vendedor(id),
    produto      VARCHAR(100),
    valor        NUMERIC(10,2)
);

INSERT INTO vendedor (nome, data_nascimento, salario) VALUES
```

```
('Ana',    '1990-05-10', 1500),
('Bruna',  '1988-03-22', 1800),
('Carlos', '1995-11-02', 1200);

INSERT INTO venda (id_vendedor, produto, valor) VALUES
(1, 'martelo', 600),
(1, 'prego',   500),
(2, 'chave de fenda', 300),
(3, 'serrote', 900);
```

a)

```
SELECT * FROM vendedor;
```

b)

```
SELECT * FROM vendedor ORDER BY nome;
```

c)

```
SELECT nome FROM vendedor WHERE nome ILIKE '%na%';
```

Resultado: Ana

d)

```
SELECT SUM(valor) AS total_vendas FROM venda;
```

e)

```
SELECT v.nome, SUM(ve.valor) AS total_vendido
FROM vendedor v
JOIN venda ve ON ve.id_vendedor = v.id
GROUP BY v.nome;
```

f)

```
SELECT nome, data_nascimento
FROM vendedor
WHERE id IN (SELECT id_vendedor FROM venda);
```

g)

```
SELECT v.nome
FROM vendedor v
JOIN venda ve ON ve.id_vendedor = v.id
WHERE ve.produto = 'martelo';
```

Resultado: Ana

h)

```
SELECT COUNT(*) AS total_vendedores FROM vendedor;
```

i)

```
UPDATE vendedor
SET salario = salario + 100
WHERE id IN (
    SELECT id_vendedor
    FROM venda
    GROUP BY id_vendedor
    HAVING SUM(valor) > 1000
);
```

Resultado: Ana (600+500=1100 > 1000) recebe o reajuste.

11.

> Utilizando as tabelas desenvolvidas no exercício anterior, elabore:
>
> a) Um exemplo de junção interna
>
> b) Um exemplo de junção externa à direita
>
> c) Um exemplo de junção externa à esquerda

Resposta:

a) Junção interna (INNER JOIN)

```
SELECT v.nome, ve.produto, ve.valor
FROM vendedor v
INNER JOIN venda ve ON ve.id_vendedor = v.id;
```

b) Junção externa à direita (RIGHT JOIN)

```
SELECT v.nome, ve.produto, ve.valor
FROM vendedor v
RIGHT JOIN venda ve ON ve.id_vendedor = v.id;
```

c) Junção externa à esquerda (LEFT JOIN)

```
SELECT v.nome, ve.produto, ve.valor
FROM vendedor v
LEFT JOIN venda ve ON ve.id_vendedor = v.id;
```

12.

> Qual palavra-chave você deve acrescentar à cláusula SELECT para assegurar que cada linha do resultado de uma busca seja única?

[ ] **a)** ALL
[ ] **b)** ROLLUP
[X] c) DISTINT
[ ] **d)** CUBE

Resposta:

A alternativa correta é a c) — a grafia no documento original está "DISTINT", mas a palavra-chave correta em SQL é DISTINCT.

13.

> SQL é uma linguagem de banco de dados abrangente. Ela inclui tanto a DDL para definição de dados, quando a DML para consultas e atualizações. Dos comandos SQL relacionados abaixo, qual não faz parte da DML?

[X] A) ALTER TABLE
[ ] **B)** INSERT
[ ] **C)** DELETE
[ ] **D)** UPDATE
[ ] **E)** SELECT

Resposta:

ALTER TABLE pertence à DDL (Data Definition Language), pois altera a estrutura da tabela, e não seus dados.

14.

> Utilizando os dados abaixo, apresente o nome do médico mais alto. Implemente uma subconsulta utilizando o predicado IN.

Médico

id	nome	altura
1	M1	1.7
2	M2	1.8
3	M3	1.9
4	M4	1.68

Resposta:

```
SELECT nome
FROM medico
WHERE altura IN (SELECT MAX(altura) FROM medico);
```

Resultado: M3 (altura 1.9)

15.

> Inseri um novo registro na tabela médico (conforme abaixo) e apresente o nome do médico mais baixo. Implemente uma subconsulta utilizando o predicado IN.

Médico

id	nome	altura
5	M5	1.68

Resposta:

```
INSERT INTO medico (id, nome, altura) VALUES (5, 'M5', 1.68);

SELECT nome
FROM medico
WHERE altura IN (SELECT MIN(altura) FROM medico);
```

Resultado: M4 e M5 (ambos com altura 1.68 — empate no valor mínimo)

16.

> Liste o nome dos médicos com consultas marcadas entre o dia 01/dez/16 a 31/dez/2016. Crie uma tabela chamada consulta com os seguintes dados e implemente uma subconsulta utilizando o predicado IN.

Consulta

id	idPaciente	idMedico	data
1	1	1	10/10/2016
2	2	1	05/12/2016
3	3	2	03/02/2017
4	1	3	15/12/2016

idPaciente referencia Paciente

idMedico referencia Medico

Resposta:

```
CREATE TABLE consulta (
    id         INT PRIMARY KEY,
    idPaciente INT REFERENCES paciente(id),
    idMedico   INT REFERENCES medico(id),
    data       DATE
);

INSERT INTO consulta (id, idPaciente, idMedico, data) VALUES
(1, 1, 1, '2016-10-10'),
(2, 2, 1, '2016-12-05'),
```

```
(3, 3, 2, '2017-02-03'),
(4, 1, 3, '2016-12-15');

SELECT nome
FROM medico
WHERE id IN (
    SELECT idMedico FROM consulta
    WHERE data BETWEEN '2016-12-01' AND '2016-12-31'
);
```

Resultado: M1 e M3

(Tabela de referência apresentada antes da questão 17)

Paciente

id	nome
1	P1
2	P2
3	P3

17.

> Acrescente à tabela paciente o campo idade e insira os seguintes dados para os pacientes:

Paciente

id	nome	Idade
1	P1	10
2	P2	22
3	P3	38

> Após, implemente uma subconsulta, utilizando o predicado IN, que mostre o nome dos pacientes com idade superior a 21 anos que têm consulta marcada.

Resposta:

```
ALTER TABLE paciente ADD COLUMN idade INT;

UPDATE paciente SET idade = 10 WHERE id = 1;
UPDATE paciente SET idade = 22 WHERE id = 2;
UPDATE paciente SET idade = 38 WHERE id = 3;

SELECT nome
FROM paciente
WHERE idade > 21
  AND id IN (SELECT idPaciente FROM consulta);
```

Resultado: P2 e P3

18.

> Utilizando uma subconsulta e o predicado ANY, mostre o nome de cada paciente, exceto o mais jovem.

Resposta:

```
SELECT nome
FROM paciente
WHERE idade > ANY (SELECT idade FROM paciente);
```

A condição "idade > ANY (...)" é verdadeira para quem tem idade maior que pelo menos um outro valor da lista — isso só é falso para o menor valor de todos (que não é maior que nenhum outro), excluindo exatamente o paciente mais jovem.

Resultado: P2 e P3 (P1, o mais jovem, é excluído)

19.

> Altera a tabela Médico. Insira dois campos: salario e idEspecialidade referencia Especialidade. Crie uma tabela especialidade com os seguintes valores:

Médico

id	nome	altura	salario	idEspecialidade
1	M1	1.7	1000	3
2	M2	1.8	5000	2
3	M3	1.9	10000	1
4	M4	1.68	12000	3
5	M5	1.68	15000	1

Especialidade

Id	nome
1	Cardiologia
2	Ortopedia
3	Pediatria

> Elabore uma subconsulta, utilizando o predicado ALL, para trazer e mostrar o nome dos médicos que recebem salários superiores aos salários pagos para os médicos com especialidade igual a pediatria.

Resposta:

```
ALTER TABLE medico ADD COLUMN salario NUMERIC(10,2);
ALTER TABLE medico ADD COLUMN idEspecialidade INT REFERENCES
especialidade(id);

CREATE TABLE especialidade (
    id   INT PRIMARY KEY,
    nome VARCHAR(50)
);

INSERT INTO especialidade (id, nome) VALUES
(1, 'Cardiologia'), (2, 'Ortopedia'), (3, 'Pediatria');

UPDATE medico SET salario = 1000,  idEspecialidade = 3 WHERE id = 1;
UPDATE medico SET salario = 5000,  idEspecialidade = 2 WHERE id = 2;
UPDATE medico SET salario = 10000, idEspecialidade = 1 WHERE id = 3;
UPDATE medico SET salario = 12000, idEspecialidade = 3 WHERE id = 4;
UPDATE medico SET salario = 15000, idEspecialidade = 1 WHERE id = 5;

SELECT nome
FROM medico
WHERE salario > ALL (
    SELECT salario FROM medico
    WHERE idEspecialidade = (SELECT id FROM especialidade WHERE nome =
'Pediatria')
);
```

Resultado: M5 (salário 15000 > maior salário da pediatria, que é 12000)

20.

> Insira um novo paciente na tabela paciente, conforme abaixo. Liste o nome dos pacientes que realizaram consultas ao menos uma vez.

Paciente

id	nome	Idade
1	P1	10
2	P2	22
3	P3	38
4	P4	44

> a) Elabore uma subconsulta utilizando o predicado EXISTS.
>
> b) Elabore a mesma subconsulta do item a) utilizando o predicado IN;

Resposta:

```
INSERT INTO paciente (id, nome, idade) VALUES (4, 'P4', 44);
```

a) Com EXISTS

```
SELECT p.nome
FROM paciente p
WHERE EXISTS (
    SELECT 1 FROM consulta c WHERE c.idPaciente = p.id
);
```

b) Com IN

```
SELECT p.nome
FROM paciente p
WHERE p.id IN (
    SELECT c.idPaciente FROM consulta c
);
```

Resultado: P1, P2 e P3 (P4 ainda não tem consulta registrada)

21.

> Considere as seguintes tabelas de um banco de dados:

• Fornecedor (cod_fornec, nome_fornec, telefone, cidade, uf)

• Estado (uf, nome_estado)

> A expressão SQL que obtém os nomes dos estados para os quais não há fornecedores cadastrados é:

[ ] **a)**

```
SELECT E.UF
FROM Estado AS E
WHERE E.nome_estado NOT IN (
SELECT F.UF
FROM Fornecedor AS F);
```

[ ] **b)**

```
SELECT E.nome_estado
FROM Estado AS E, FROM
Fornecedor AS F
WHERE E.UF = F.UF;
```

[X] c)

```
SELECT E.nome_estado
FROM Estado AS E
WHERE E.UF NOT IN (
```

```
SELECT F.UF
FROM Fornecedor AS F);
```

[ ] **d)**

```
SELECT E.nome_estado
FROM Estado AS E, FROM
Fornecedor AS F
WHERE E.nome_estado = F.UF;
```

[ ] **e)**

```
SELECT E.nome_estado
FROM Estado AS E
WHERE E.UF IN (
SELECT F.UF
FROM Fornecedor AS F);
```

Resposta:

A alternativa c) está correta (é a alternativa marcada com X no documento original): compara UF com UF (mesmo domínio) e usa NOT IN para trazer os estados que não aparecem na lista de UFs de Fornecedor. As opções b) e d) têm um erro de sintaxe (duplo FROM / múltiplas tabelas sem JOIN correto), e a opção e) traz o resultado invertido (estados COM fornecedores).

22.

> Pesquise o significado das siglas DDL, DML, DQL, DTL e DCL crie uma tabela com exemplos de cada comandos em SQL associado à sigla. Implemente cada exemplo no PostgreSQL.

Resposta:

Sigla	Significado	Comandos	Exemplo em PostgreSQL
DDL	Data Definition Language (Linguagem de Definição de Dados)	CREATE, ALTER, DROP, TRUNCATE	CREATE TABLE produto (id SERIAL PRIMARY KEY, nome VARCHAR(50));
DML	Data Manipulation Language (Linguagem de Manipulação de Dados)	INSERT, UPDATE, DELETE	INSERT INTO produto (nome) VALUES ('Martelo');
DQL	Data Query Language (Linguagem de Consulta de Dados)	SELECT	SELECT * FROM produto;
DTL / TCL	Data/Transaction Control Language (Linguagem de Controle de Transação)	COMMIT, ROLLBACK, SAVEPOINT	BEGIN; UPDATE produto SET nome = 'Serrote' WHERE id = 1; COMMIT;
DCL	Data Control Language (Linguagem de Controle de Dados)	GRANT, REVOKE	GRANT SELECT ON produto TO usuario_leitura;